



02

What Companies Are Looking For

Technical skills alone don't determine your offer. Otherwise, those who can solve the coding and system design problems would get the same result. Instead, companies use behavioral interviews to answer two critical questions: *Do you fit with both the role and the company?* And if you do fit, *at what level will you be most effective?*



Get both right, and you will receive an offer at the appropriate level. Get the fit wrong, and you'll be rejected regardless of your skills. Get the level wrong, and you'll be either down-leveled or rejected for being underqualified.

This chapter explains how companies make their assessments of fit and level by analyzing the signals in your stories. Once you understand these dimensions, you'll pick better stories and signal the right level.

Understanding Fit: Role and Company

The primary consideration for any tech role is whether you have the technical skills to do the job. Companies will assess this mostly through the technical parts of the interview, for example, coding challenges, system design, or whatever technical evaluation matches your role. If you can't demonstrate the core technical capability, nothing else matters.

But technical skills alone don't predict success. Companies learned this the hard way by hiring smart people who couldn't work effectively in their environment. That's why behavioral interviews focus on two additional types of fit:

🔗 Role Fit: Can you handle the specific challenges and working conditions of this position? A backend role at a fast-growing startup requires different capabilities than a backend role at an established enterprise. The technical skills might be similar, but the role demands will be different.

making approach, and values match with how the company gets things done.

How Companies Detect Fit Through Signals

Companies can't directly ask the question, "Would you fit here?" What candidate would torpedo their chance of success by answering with a "No"? Instead, companies look for signals in your stories that indicate alignment or misalignment.

Role Fit Signals emerge from how you describe handling situations similar to what the role requires:

- If the role requires working with ambiguous requirements, do your stories show comfort with uncertainty?
- If the position involves cross-team coordination, do you show an ability to cope with organizational complexity?
- If the job needs rapid iteration, do your examples show shipping quickly and adjusting based on feedback?

Company Fit Signals come from the choices you made and how you describe them:

- A company that values "bias for action" looks for stories that show you moving quickly despite incomplete information.
- An organization that prizes "customer obsession" wants to hear examples of you going deep to understand user needs.
- A place that emphasizes "radical transparency" seeks stories that show you sharing information openly, even when you're uncomfortable.

The same story can send different signals to different companies. You spending three weeks perfecting a solution might demonstrate attention to quality at one company but analysis paralysis at another. Moving fast and fixing issues later demonstrates good judgment at a growth startup but recklessness at an established healthcare company.

Common "Mis-Fits"

Even a talented candidate will get rejected sometimes if they are not a good fit. The same behaviors that are positive at one company can signal poor fit at another.

Independence vs. Collaboration: This covers both how you work and how you make decisions. Some companies need people who pick up a problem, run with it, and come back with a solution. Others expect you to bring the team along at every step. These often go together: companies that want you to work solo also tend to want you to make calls on your own, and companies that want collaborative work also want group buy-in on decisions.

checking with the group before you act, companies that prize individual ownership will wonder whether you can make a decision without a meeting.

Speed vs. Thoroughness: Startups often need rapid experimentation, where you ship MVPs and iterate based on feedback, while companies in healthcare or finance require careful validation before any release. This tension also shows up in how teams think about code quality: some organizations will happily spend extra weeks on clean architecture, while others want a working solution on deadline even if the code needs cleanup later. Whereas stories about methodical testing might bore a startup, your "ship it and fix it" examples could terrify a medical device company.

Excellence vs. Pragmatism: Some organizations value technical excellence and clean architecture above all else. Others need pragmatic solutions that ship on deadline even if imperfect. Focusing on perfect code fails at deadline-driven companies, just as accepting technical debt everywhere fails at companies maintaining critical infrastructure.

Innovation vs. Stability: Some roles require creating new solutions and challenging existing approaches, while others need you to maintain and optimize proven systems. If you say that you're constantly reinventing established processes, teams that value stability will not consider you a good fit. Conversely, stories that show you only follow existing patterns will disappoint teams that are looking for creative problem-solving.

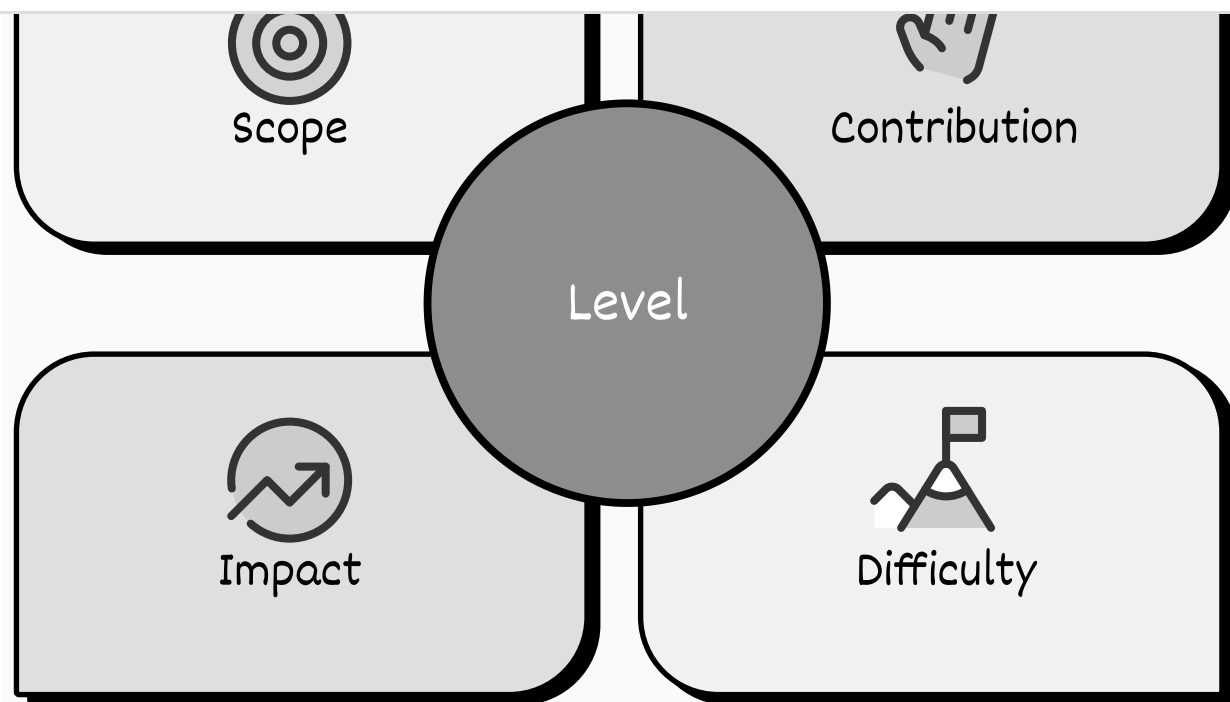
Direct vs. Diplomatic: Some cultures prize radical candor and want you to say exactly what you think. Others value maintaining harmony and face-saving communication. If you are too blunt, you will not fit in well at a relationship-focused company. If you are not direct enough, you will not like working at a company that values "disagree and commit."

Data vs. Intuition: Some companies require data to justify every decision ("data-driven" cultures), while others trust experienced judgment and move on gut feel. Showing that you make decisions based on instinct does not impress analytical companies, and telling a company that values experienced judgment that you conduct three A/B tests to choose a button color will get you struck off their list.

Specialist vs. Generalist: Large companies often want deep experts who master one domain, while smaller companies need people who are comfortable wearing multiple hats. Know which sort of company you are walking into.

Once you understand fit, you can pick stories that match the company and the role.

The Four Dimensions That Determine Your Level



Companies assess your level through four dimensions that appear in every story you tell. Each dimension reveals different aspects of your capability. Together, they show the company where you operate most effectively.

🎯 Scope

Scope provides a measure of the number of people on your team and, extending outward as you advance, whose work was affected by your actions. The greater the number affected, the higher your level for this dimension.

Entry Level: Your work affects your own productivity and starts to help other team members. For example, you might improve how you handle assigned tasks or fix issues that were slowing down a few teammates.

Mid Level: Your work affects aspects of the team and shapes how it operates. You might redesign a process that changes a significant part of how your team works or solve problems that affect most of the team's effectiveness.

Senior Level: Your work directly impacts your entire team and is beginning to influence at least one other team. Perhaps you create solutions that change how your whole team operates and affect workflows in adjacent teams, or you solve problems that require coordination with other groups. You may also start collaborating more closely with product or design partners on your immediate team's work.

how multiple teams make decisions and solving problems that require buy-in across several parts of engineering. Your influence extends beyond engineering into product, design, and program management as you shape solutions that affect how cross-functional partners work.

Principal Level: Your work affects many teams or changes how large parts of the organization operate. Perhaps you have created technical strategies that have influenced how dozens of teams make decisions. Or you have solved problems that cut across a large engineering organization. At this level, your influence regularly extends into business strategy, shaping decisions alongside product, design, program, and business leadership.

Contribution

Contribution captures what *you* did, not what happened around you. It is important to be precise about the line between "I" and "we." Companies will expect to see evidence of increasing leadership and ownership as you advance in your career.

Entry Level: You execute assigned work and are beginning to take ownership of small pieces. Examples: implementing solutions designed by others; fixing bugs in existing systems; taking full responsibility for well-defined features within larger projects.

Mid Level: You own complete solutions from problem to implementation while also guiding others. Perhaps you have identified issues, designed the approaches, implemented them, and you have verified that they work, and you have helped your teammates understand the reasons for your decisions.

Senior Level: You lead initiatives requiring coordination. You're expected to make progress even when the requirements are unclear or the path forward is uncertain. Examples of this include driving technical decisions for your team; mentoring others through complex problems; architecting solutions to be implemented by others; and ensuring quality work outcomes for many people.

Staff Level: You lead cross-team initiatives and establish technical direction, often in situations where the right approach isn't obvious and stakeholders have competing priorities. This could look like defining technical approaches that are adopted by multiple teams, creating systems that enable other teams to solve problems on their own, or driving agreement on complex technical decisions across several teams.

Principal Level: You create organizational capabilities and establish new ways of working. At this level, you're frequently operating in highly ambiguous environments where you must define the problem before you can solve it. You might define technical standards that guide dozens of teams, build

Impact

Impact shows what changed for the better as a result of your work. Companies want to see that your work produced results worth the investment. Strong stories put numbers on the impact and connect technical wins to business or user outcomes.

Entry Level: You improve your personal productivity and are starting to help the team work better. Examples include reducing the time you spend on repetitive tasks, fixing issues that were slowing down teammates, or improving the quality of code in the areas you touch. Even simple measures matter at this level: time saved or bugs prevented.

Mid Level: You improve team effectiveness in specific areas and influence team-wide practices. Perhaps you reduced deployment times for specific workflows, eliminated categories of bugs in your domain, or you created tools that have made the team more productive in particular areas. You can quantify these improvements and connect them to broader outcomes like feature velocity or reliability.

Senior Level: You transform how your entire team works and are starting to have an impact beyond your team. For example, you might have introduced new workflows that changed your team's capabilities. Or perhaps you eliminated major sources of operational problems, or the improvements that you have created have been adopted by adjacent teams. Your impact extends beyond just engineering metrics to product outcomes, user experience, or operational costs.

Staff Level: You improve how multiple teams operate and drive organizational improvements. These sorts of impact come from achievements such as establishing practices that several teams adopt, solving infrastructure problems that were impeding multiple teams, or creating new capabilities that open up new types of work across teams. Your measurable impact can be tied to business metrics like revenue, customer retention, or time-to-market.

Principal Level: You create organizational capabilities and drive strategic changes. Impact at this level could come from establishing technical foundations that dozens of teams use to build upon, solving problems that were blocking major business initiatives, or creating leverage that compounds benefits across the company. Your impact is measured in business outcomes and strategic capability, not just technical improvements.

Difficulty

less impressive than hard problems solved well.

Entry Level: You work on straightforward problems within established patterns. For example, you might face challenges learning new technologies or debugging unfamiliar code, but the path forward becomes clearer once you understand the problem or ask for help.

Mid Level: You work through challenges and obstacles in your work. The problems you tackle have more moving parts and less obvious solutions. These could be competing requirements or having to work through technical complexity you haven't seen before. Or perhaps you have had to manage dependencies within your team that affected your timeline or figure out solutions when the approach wasn't immediately obvious.

Senior Level: You manage constraints and make technical decisions with team-level architectural implications. The problems you solve involve multiple interacting systems and competing concerns. You might have to balance needs across multiple stakeholders with different priorities. Maybe you make architectural decisions that affect how your whole team works, or you have to work around technical limitations that require creative solutions, or solve problems that require you to address both technical and business factors.

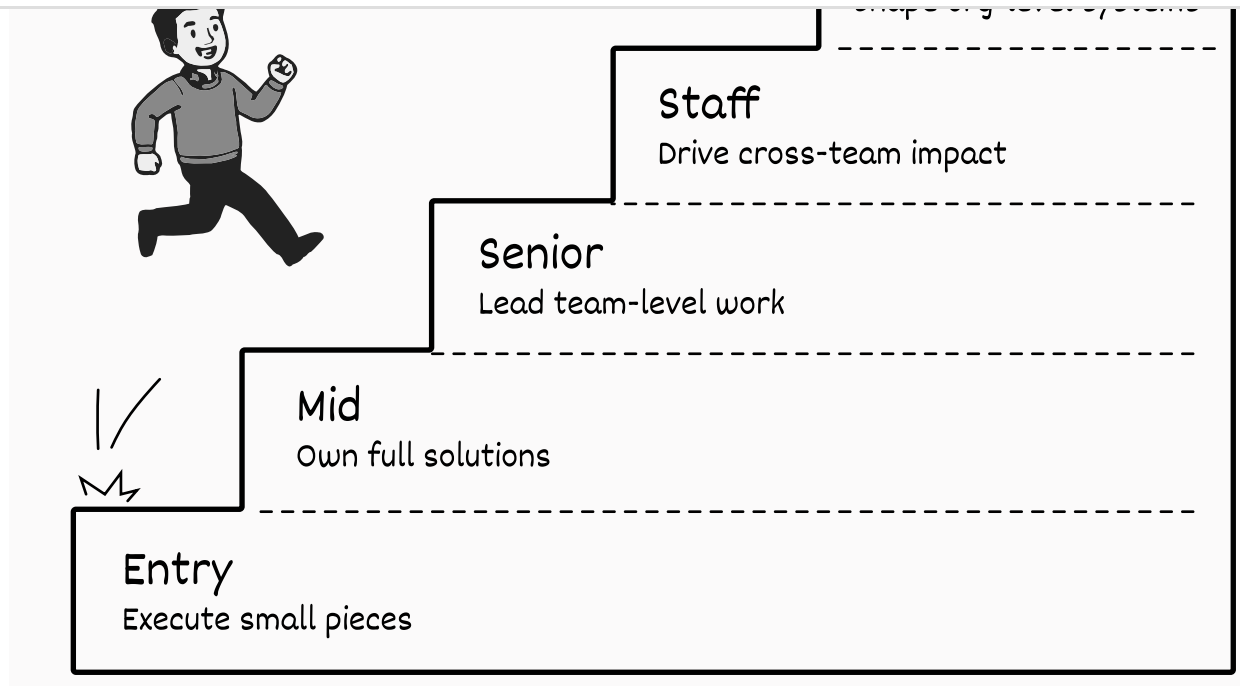
Staff Level: You manage competing trade-offs across multiple teams while handling problems with significant technical and organizational complexity. Examples of difficulty at staff level include:

- Balancing different technical approaches when teams have genuinely conflicting needs.
- Creating solutions that affect how several teams work together.
- Making architectural decisions that have to work across diverse contexts.
- Getting teams to agree when the technically optimal solution differs for each team.

Principal Level: You handle fundamental trade-offs between competing organizational needs or solve problems where no clear solution exists. The complexity at this level often involves novel problems that lack established patterns or precedents. You might balance technical excellence against delivery speed at organizational scale; work within organizational constraints while maintaining technical integrity; create approaches for entire classes of problems the company hasn't solved before; or make decisions that affect company strategy and require executive buy-in.

What Each Level Looks Like

Here's how the same types of accomplishments look across each level. These aren't templates. They're meant to help you develop a sense for the difference between a mid-level story and a senior one. Compare adjacent levels and notice what actually changes as you move up and down.



Entry Level Through the Four Dimensions

Story: Improved testing workflow

- *Scope*: My testing work, and three teammates who handled similar tasks
- *Contribution*: I wrote the scripts and documented the process
- *Impact*: Manual testing went from a 2-hour chore to 30 minutes
- *Difficulty*: Needed to learn our testing framework and understand how my teammates used it

Story: Fixed recurring production alerts

- *Scope*: My team's on-call rotation
- *Contribution*: I investigated the root cause of the alerts and implemented the fix
- *Impact*: Eliminated roughly 15 alerts per week that were waking people up
- *Difficulty*: Had to trace through unfamiliar code and test thoroughly to avoid breaking existing functionality

Story: Created onboarding documentation

- *Scope*: New people joining my team, and improving our documentation practices
- *Contribution*: I documented our setup process, ascertained the common issues, and worked with my manager to review accuracy
- *Impact*: New hires were productive in two days instead of a week, and my docs became the template that other teams copied
- *Difficulty*: Had to identify which parts were confusing and work with my manager to verify accuracy

- *Scope:* My team's deployment workflow, and starting to influence the infrastructure team's approach
- *Contribution:* Redesign, implementation, and rollout
- *Impact:* Reduced deployment time from 3 hours to 15 minutes, enabling daily releases instead of weekly
- *Difficulty:* Had to balance speed with reliability, manage dependencies within the team during migration, and ensure nothing broke while rolling out changes

Story: Built automated performance testing

- *Scope:* All the backend services that my team maintained
- *Contribution:* Designed the framework and implemented the initial version
- *Impact:* Caught performance regressions before production; reduced incidents by about half
- *Difficulty:* Had to balance test coverage with execution time, figure out integration with existing CI/CD without clear documentation, and handle flaky tests

Story: Improved error handling across services

- *Scope:* My team's microservices, and influencing how we thought about observability
- *Contribution:* I owned the strategy, implemented it across our services, and documented the patterns
- *Impact:* Debugging that used to take hours now took minutes; customers saw fewer and shorter outages
- *Difficulty:* Had to balance detail with performance impact, coordinate changes across six services my team owned, and handle edge cases that weren't well documented

Senior Level Through the Four Dimensions

Story: Established shared component library

- *Scope:* My entire team and three other frontend teams
- *Contribution:* I drove the technical vision, built initial buy-in, and led implementation
- *Impact:* Less duplicate code, more visual consistency, and feature work that used to take three sprints started finishing in two
- *Difficulty:* Had to build consensus across teams with different needs, establish governance model, and migrate existing code

Story: Redesigned incident response process

- *Scope:* My team's on-call practices; and influencing two other engineering teams
- *Contribution:* Identified the problems, designed new workflows, and led adoption

- *Difficulty:* Had to change the established practices my team relied on, balancing needs across different stakeholders and making architectural decisions about communication patterns between teams

Story: Architected service communication pattern

- *Scope:* My team's services and the two teams that integrated with us
- *Contribution:* Designed the architecture, led technical implementation, and ensured adoption
- *Impact:* Teams could deploy independently; reduced integration bugs by about 70%
- *Difficulty:* Required making architectural decisions that affected how all three teams would build features going forward, balancing different team constraints, and technical integration across systems with different patterns

Staff Level Through the Four Dimensions

Story: Created platform observability standards

- *Scope:* Three backend teams directly; influenced broader engineering organization
- *Contribution:* Defined the technical strategy, built the initial implementation, and drove adoption
- *Impact:* Reduced mean time to detect issues from 30 minutes to under 2 minutes; teams could debug on their own without waiting for the platform team
- *Difficulty:* Required hard trade-offs between standardization and team autonomy; teams had different observability needs based on their systems; had to create an approach that worked across diverse technical contexts

Story: Established API design framework

- *Scope:* Five product teams building customer-facing APIs
- *Contribution:* Led the technical design, built reference implementations, and created governance model
- *Impact:* Partners could integrate in half the time they used to, API-related support tickets dropped, and the APIs started to feel like one product instead of five
- *Difficulty:* Teams had directly conflicting requirements based on their different use cases; had to balance flexibility against consistency when both needs were valid; created framework that worked across internal and external APIs with different constraints

Story: Transformed deployment infrastructure

- *Scope:* Backend and infrastructure teams (about 40 engineers across 4 teams)
- *Contribution:* Designed the architecture; coordinated implementation across teams; drove organizational adoption

- *Difficulty*: Each team had its own different deployment needs based on their system characteristics; required trade-offs between deployment speed and safety across different contexts; had to design architecture that worked for both monoliths and microservices

Principal Level Through the Four Dimensions

Story: Created engineering strategy for AI infrastructure

- *Scope*: All teams building AI-powered features (15+ teams)
- *Contribution*: Defined the technical strategy and built organizational support
- *Impact*: Enabled company to ship AI agents in days instead of weeks or months; this became a competitive advantage
- *Difficulty*: Required deep technical trade-offs between flexibility, security, and standardization; executive alignment; multi-quarter rollout

Story: Transformed how company handles data privacy

- *Scope*: Entire engineering organization and product teams
- *Contribution*: Established technical frameworks and governance model
- *Impact*: Achieved compliance requirements; enabled new markets; built trust with users
- *Difficulty*: Managed legal requirements and organizational constraints across product and legal teams; balanced product velocity with compliance at scale; required executive alignment on trade-offs between features and privacy

Story: Established technical fellowship program

- *Scope*: Senior and staff engineers across the company
- *Contribution*: Designed the program structure and mentored initial fellows
- *Impact*: Developed next generation of technical leaders; improved retention of senior talent
- *Difficulty*: Required working around organizational constraints around career development and compensation; built support from executive leadership; balanced program overhead with organizational value when no clear ROI metrics existed

Reading and Calibrating Your Own Level

Use these dimensions to check whether your stories match your target level. Most candidates either undersell their accomplishments or overstate their contributions. This section will help you find the more accurate middle ground.

Assessing Your Stories

Take your prepared stories and score them using this table:

 Story Topic	 Scope Who was affected?	 Contribution What did You do?	 Impact What changed?	 Difficulty What made it hard?	 Overall Level
Example: Fixed broken deployment process	Just my team (Mid)	Led the entire fix (Senior)	3 hours → 15 mins deploy time (Mid)	Had to verify no downstream impact with 3 teams (Mid/Senior)	Mid-Senior

Scoring Guide:

- **Entry:** Individual/few teammates, executed/owned small pieces, personal/small group productivity, straightforward problems
- **Mid:** Team aspects, owned solutions/guided others, specific team improvements with emerging business connection, challenges with more moving parts and less obvious solutions
- **Senior:** Whole team + 1-2 other teams, led initiatives/mentored, transformed team + influenced others, including cross-functional partners, constraints and team-level architecture, progress despite unclear requirements
- **Staff:** 2-3+ teams directly, cross-team leadership, multi-team capabilities tied to business metrics, significant technical and organizational complexity, competing stakeholder priorities
- **Principal:** Multi-org/company-wide, strategic leadership alongside product/design/business partners, organizational capabilities measured in business outcomes, novel problems without established patterns, defining problems before solving them

Interpreting Your Results

If most stories cluster below your target level: You need to either find stronger examples from your experience, better articulate the scope and impact of your existing stories, or consider if you're targeting the right level.

If your stories show mixed levels: This is normal, but ensure you have at least 2-3 stories solidly at your target level. Lead with those in interviews. A strong senior story followed by a solid mid-level story is fine. Three mid-level stories with senior-level framing is not.

- Weak on Impact? Find stories with quantified business outcomes
- Weak on Difficulty? Choose examples with appropriate challenges for your level, for example, obstacles and complexity for mid-level, constraints and architectural decisions for senior, fundamental trade-offs for staff and above
- Weak on Contribution? Find stories where you can clearly articulate *your* specific decisions and actions, not team outcomes.

You don't need every story to be at your target level, but you need enough of them to be at that level to show you can consistently operate there.

To be clear: you're selecting which experiences to share and which aspects to emphasize, not fabricating stories. If you pretend to have experiences you don't have, you'll either get rejected when follow-up questions reveal the truth or, worse, get hired into a role you can't actually do.

The same project can be told in a multitude of ways, all truthful. Your API redesign project could emphasize technical depth, customer impact, cross-team coordination, or fast delivery, and all four might be accurate.

Tailor your emphasis to what each role values, not your underlying experiences. If your experiences don't match with what a company values, that's useful information. Maybe this isn't the right role for you. Maybe you need to acquire those experiences before applying. Or maybe you simply need to look for a different culture to work in. Far better to know this before accepting an offer than discovering the mismatch after starting.

Common Mismatches

Watch for the following patterns, which can lead to down-leveling or rejection:

Junior Stories for Senior Roles: Focusing on individual task completion when interviewing for roles requiring team leadership. For example, saying, "I optimized our search query from 3 seconds to 200ms" without mentioning how you influenced the team's approach to performance, taught others the optimization techniques, and created monitoring to prevent future degradations.

Mid-Level Stories for Staff Roles: Focusing on single-team ownership when interviewing for roles requiring cross-team leadership. For example, saying, "I owned our team's entire deployment pipeline, and it was adopted by one other team." Have you got a story that describes how you built consensus across multiple teams or created systems that let other teams solve similar problems without your help?

Claiming Unearned Scope: For example, saying that you led the entire platform rebuild when, in fact, you built only one service within a larger effort. Or "I redesigned our service architecture" when all you

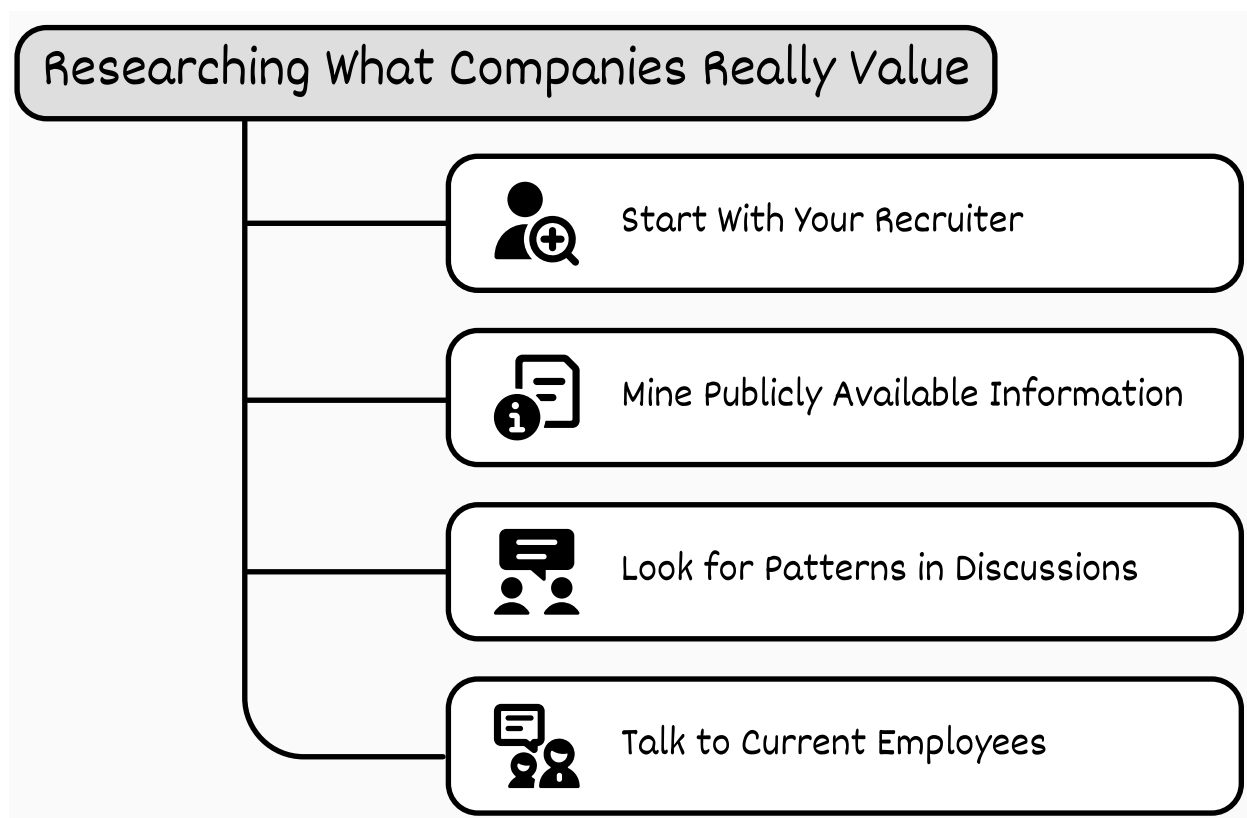
All Impact, No Difficulty: Stories about important but uncomplicated work don't demonstrate senior-level problem-solving. "I added SSL certificates to our API endpoints, improving security for 100K users." Important? Yes. Difficult? No. You're describing a straightforward implementation. For senior roles, you need to provide examples of working through constraints and making architectural decisions, not just executing important checklist items.

All Difficulty, No Impact: Deep technical work that didn't create meaningful outcomes hints at overengineering. Don't tell them "I spent three months building a custom distributed cache with eventual consistency guarantees" without explaining why existing solutions wouldn't work or what business problem this solved. Complexity without purpose raises red flags.

Unclear Contribution: Using "we" throughout your story makes it impossible to assess your level. "We migrated to Kubernetes, and we reduced deployment time by 90%." Who made the decision? Who did the work? Who handled the problems? If you are not clear about your specific role, interviewers won't be able to evaluate whether you drove the work or were just a participant.

Researching What Companies Really Value

You'll never have perfect information about what a specific company values, but a little focused research will often reveal surprising insights that most other candidates will miss. The difference between having even partial intelligence and going in blind can be whether or not you emphasize the right things in your stories.



source of insider information. Recruiters want you to succeed, because their performance is based on the number of accepted offers received by the candidates they put forward. They have prep materials, they know the interviewers' focus areas, and they understand what they are looking for.

Ask your recruiter directly: "What should I know about this company's current challenges?" Or "What competencies matter most for this role?" Or "Can you share any interview prep materials?" Many recruiters have documents about interview format, team priorities, or even the specific behavioral competencies they evaluate. The questions that are used as examples in the prep materials have a high likelihood of being asked in the interviews.

Mine Publicly Available Information

When companies repeat certain words when describing job opportunities, they're telling you what matters. For example, a job posting that mentions "fast-paced" several times signals something different than one emphasizing compliance. Those words are there for a reason.

Where to dig:

- **Engineering blogs:** How do they describe their wins? What problems do they celebrate solving?
- **Tech talks and conferences:** What topics do their engineers present? Speed of delivery? Scale? Innovation?
- **Open source contributions:** What they choose to open source reveals their priorities. If they open source developer tools, this suggests they value community. If they are happy to make internal tools public, this shows transparency.
- **Technical documentation:** The existence of public API docs or technical guides (and the quality thereof) shows how they support both users and their own teams.
- **Status pages and postmortems:** Companies that publish detailed postmortems demonstrate that they value learning from failure. A company that shares their incident response processes likely has a strong operational culture.

Even companies without engineering blogs will leave traces. Product release patterns tell you about their development pace. Technology choices show their priorities: newer frameworks suggest a focus on innovation, whereas relying on proven technologies indicates they prefer stability.

Look for Patterns in Discussions

Glassdoor, Blind, and Reddit contain gold buried amongst rubble. Ignore the rubble (e.g., individual rants). Instead, look for patterns across multiple posts. If five different people mention "lots of process" or "no work-life balance" or "amazing learning culture," that's a pattern you will want to know about.

that productivity within the company is inhibited by an excessive number of meetings. Praise for "autonomy" indicates they trust their people to make decisions without checking in. Both types of comments reveal what behaviors the companies will reward.

Talk to Current Employees

If you know someone at the company, ask them directly what behaviors get rewarded and, conversely, what behaviors will cause people to struggle. Skip surface-level queries about culture, and ask specific questions:

- "When someone gets promoted here, what do they do to earn it?"
- "What behaviors get negative feedback?"
- "How does the team make decisions when there's disagreement?"
- "What surprised you most about working here?"

Current employees will tell you truths the company website never would. Perhaps they'll tell you that at their company, "customer obsession" really means checking usage data before writing code, or that "ownership" means being available to resolve production issues at two o'clock in the morning.

What You're Really Looking For

All this research serves one purpose: understanding what stories will resonate at your interview. Think of it as finding the real intersection between your experience and what they care about.

If research reveals they prize speed over perfection, then emphasize stories that tell how you shipped quickly and iterated. If they value technical depth, highlight examples of diving deep to understand root causes. If they care about collaboration, make sure your story focuses on cross-team work rather than solo accomplishments.

The research will also help you decide whether this company is the right place for you. If everything you learn suggests they value the kinds of behaviors you don't naturally demonstrate or don't want to develop, then perhaps you don't need to pursue that particular role.

Putting It All Together

Companies aren't just evaluating whether you can do the job. They're also assessing whether you'll thrive in their specific environment and at what level you'll be most effective. These two dimensions determine not just whether you will get an offer, but also whether that offer will position you for success.

Understanding fit helps you know which of your experiences will connect most with what the company values. This small company needs someone who ships fast and figures things out alone. That

Understanding levels helps you position your stories appropriately. The same project can demonstrate entry-level execution, mid-level ownership, or senior-level leadership depending on your actual contribution and how you frame it. Get this wrong and you will either get rejected for overreaching or down-leveled for not properly communicating your capabilities.

The payoff is immediate. You'll pick better stories, focus on the right details, and make it easier for interviewers to see what you can do. You'll make better decisions about which roles actually match who you are and what you want to do. The goal isn't to get *any* offer. The goal is to get the *right* offer at the *right* level at the *right* company to ensure your success.

☐ Mark as Complete[Teach on ByteByteGo](#)[Be an Affiliate](#)[Become a Contributor](#)hi@bytebytego.com[Report a Bug](#)[Our Team](#)[Newsletter](#)[Privacy Policy](#)[Terms of Service](#)[Do Not Sell or Share My Personal Information](#)[YouTube](#)[Visual Dev Guides](#)



Prepare for Coding Interviews

Copyright ©2022-2026 ByteByteGo Inc.
All rights reserved.